

A Reproducible Data Analysis Workflow

How to lay out a project, when to use a script or a document/notebook, and how one source file becomes every report you need

Class: **dsml** · Week 1

Richard Dien Giau Bui · IB, NCCU · 2026

Most analyses cannot be rerun six months later, for boring and preventable reasons

Why it breaks

The file paths are personal:

C:/Users/Jia/Desktop/... exists on one laptop

The run order is lost: which script came first?

The raw data was overwritten: by the cleaning step

The packages moved on: the code runs, the numbers differ

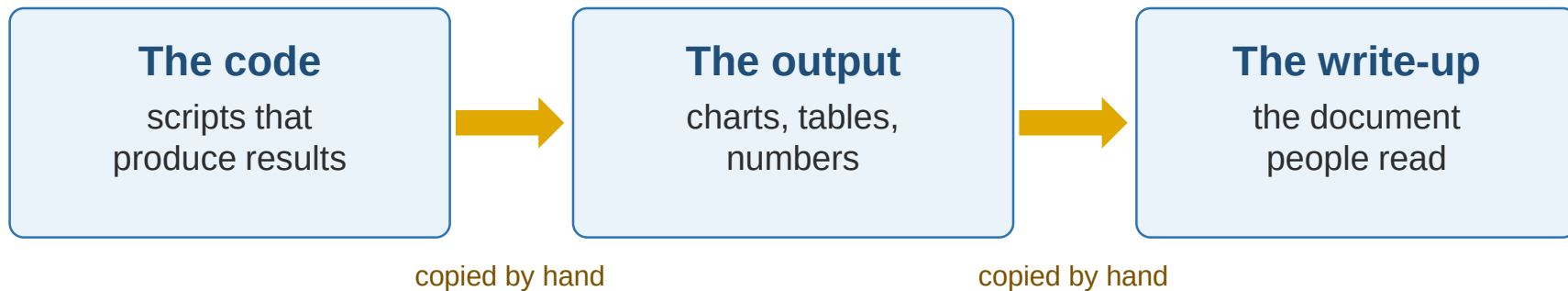
The report was assembled by hand: and nobody remembers how

What it costs

- You cannot answer "where did this number come from?"
- A teammate cannot pick the work up from you
- A correction means redoing the analysis from scratch
- The result cannot be defended to a client or an examiner

None of these are hard problems. They are all decided in the first ten minutes of a project, and almost never revisited.

Results usually live in three places that quietly drift apart



Every hand-copied number is a number that will be wrong the next time the analysis changes.

The fix is to delete the manual arrows. The report should be generated from the code, not assembled alongside it — which is exactly what Part 3 is about.

Today's question: how do you set up a project so it can be rerun and the report rebuilt, by anyone?

Someone clones your project, opens it, runs one command — and gets your exact results and your finished report.

By the end of this class you will be able to

- Lay out a project folder that a stranger can navigate without asking you
- Decide when a task belongs in a script and when it belongs in a document
- Write a Quarto document that mixes prose, code, and results in one file
- Render that one file to HTML, PDF, Word, or slides without rewriting it

We work outward: the folder, then the file, then the output

1

The folder

Where things live, and why the layout is a decision worth making once

2

The file

Script or document? What each is good at, and why most projects use both

3

The output

Markdown, and how one source file becomes every report format you need

Part 1

The folder

Where things live, and why the layout is a decision worth making once

A reproducible project is one self-contained folder with one obvious entry point

Self-contained

- Everything the analysis needs is inside the folder
- Nothing depends on where the folder sits on disk
- Copy it, zip it, or clone it and it still runs

One obvious entry point

- A README that says what to run, in what order
- One .Rproj file, so paths resolve from the root
- Scripts named so the order is visible at a glance

The test: hand the folder to a classmate who has never seen it. If they need to ask you a single question before running it, it is not finished.

This folder is also the unit of version control — one project, one repository, as in last week's session.

One layout works for almost every analysis project you will do

```
sales-forecast/  
  README.md  
  sales-forecast.Rproj  
  .gitignore  
  data/  
    raw/          <- never edited  
    process/      <- regenerated  
  R/              <- helper functions  
  scripts/  
    01_clean.R  
    02_model.R  
  outputs/        <- figures, tables  
  report.qmd      <- the deliverable
```

The three-way split

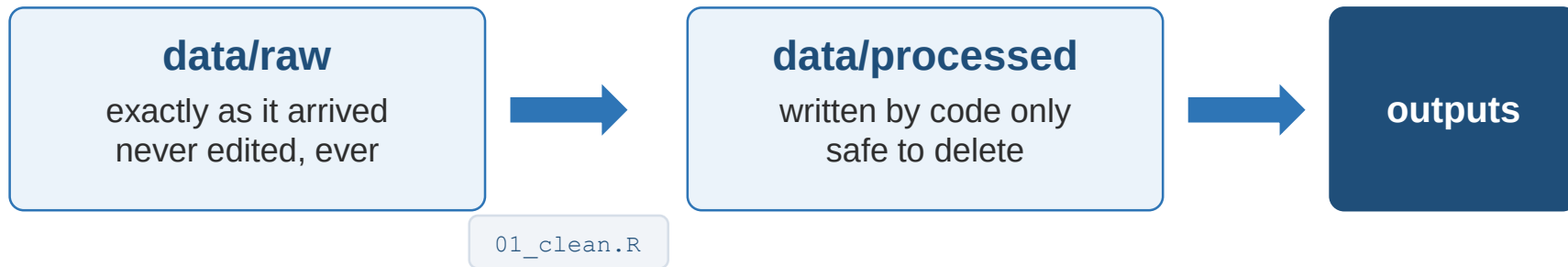
Inputs: data/raw, never touched by code

Process: R/ and scripts/, all your logic

Outputs: outputs/ and the report, all regenerated

Anything in outputs/ can be deleted and rebuilt by rerunning the code. That is the test for whether it belongs there.

Raw data is read-only — every other file is something the code can rebuild



If you fixed something by editing the spreadsheet, the fix is invisible and cannot be repeated. Every correction belongs in a script.

In practice

- Set the files in data/raw to read-only in your operating system
- Never open a raw file in Excel and press save
- If the raw data is large or confidential, gitignore it and describe it in the README

Absolute paths are the most common reason code fails on another machine

Breaks on every other machine

```
setwd("C:/Users/Jia/Desktop/proj")  
read.csv("C:/Users/Jia/Desktop/  
proj/data/raw/sales.csv")
```

Works everywhere

```
library(here)  
read.csv(here("data", "raw",  
"sales.csv"))
```

Why here() works

It finds the project root itself: by locating the .Rproj file, wherever the folder sits

It handles both slash conventions: so Windows and Mac users share one script

It does not care where you launched R from: the path is always built from the root

setwd() should never appear in a script you intend to share. Open the .Rproj file instead.

Number your scripts so the run order cannot be guessed wrong

Name by order, then by job

```
scripts/  
  00_setup.R  
  01_clean.R  
  02_model.R  
  03_figures.R
```

Why it matters

- The order is visible without opening anything
- A new teammate needs no explanation from you
- The numbers never fall out of date, unlike comments

The goal: one command rebuilds everything

```
# main.R  
source(here::here("scripts", "01_clean.R"))  
source(here::here("scripts", "02_model.R"))
```

If rebuilding the project takes one command, it is reproducible. If it takes instructions, it is not.

The README and the lockfile are what make the folder genuinely self-contained

README.md — for the human

- What the project does, in one sentence
- What has to be installed before it runs
- Which script to run, in what order
- Where the data came from, and any access limits

renv.lock — for the machine

- Records the exact package versions you used
- `renv::init()` starts it; `renv::snapshot()` updates it
- A collaborator runs `renv::restore()` to match you
- Commit `renv.lock`, gitignore `renv/library/`

Without a lockfile the code still runs a year later — it just quietly returns different numbers, because a package changed underneath it. That is the failure that is hardest to spot.

Part 2

The file

Script or document/notebook? What each is good at, and why most projects use both

A script computes; a document/notebook computes and explains in the same file

A script: script.R

```
# clean the sales data
library(here)
sales <- read.csv(
  here("data", "raw", "sales.csv"))
mean(sales$revenue)
```

Code is the default. Prose survives only as comments, and the results appear in the console.

A document: report.qmd/report.ipynb

```
## Revenue in 2025

Average monthly revenue was:

```{r}
mean(sales$revenue)
```
```

Prose is the default. Code sits in chunks, and the results appear in the finished document.

Same computation, different job. The script produces a number; the document produces something a reader can follow.

Each format is better at a different part of the work

| Consideration | R script (.R) | Quarto document (.qmd) |
|----------------------------|--------------------------------|--------------------------------------|
| Main content | Code | Prose, with code in chunks |
| Where results appear | Console and files | Inside the finished document |
| Explaining your reasoning | Comments only | Full formatted text |
| Running part of the work | Line by line, easy | Chunk by chunk, dependable & riskier |
| Being called by other code | Easy: <code>source()</code> it | Not designed for it |
| Producing a deliverable | No, you assemble it yourself | Yes, it renders one |

Neither one wins. The question is what the output of this particular task is meant to be.

Choose by deliverable: a script when the output is data, a document when it is the report

Use a script when

- The output is a cleaned dataset or a model
- The step will be rerun automatically
- Other code needs to call it
- The work is long and needs debugging

Use a document when

- The output is something a person reads
- The reasoning matters as much as the result
- Charts and tables belong beside the text
- You will produce it again with new data

The question to ask: what does this piece of work hand to the next step — a dataset, or a reader?

The notebook trap: chunks run out of order produce results you cannot reproduce

How it happens

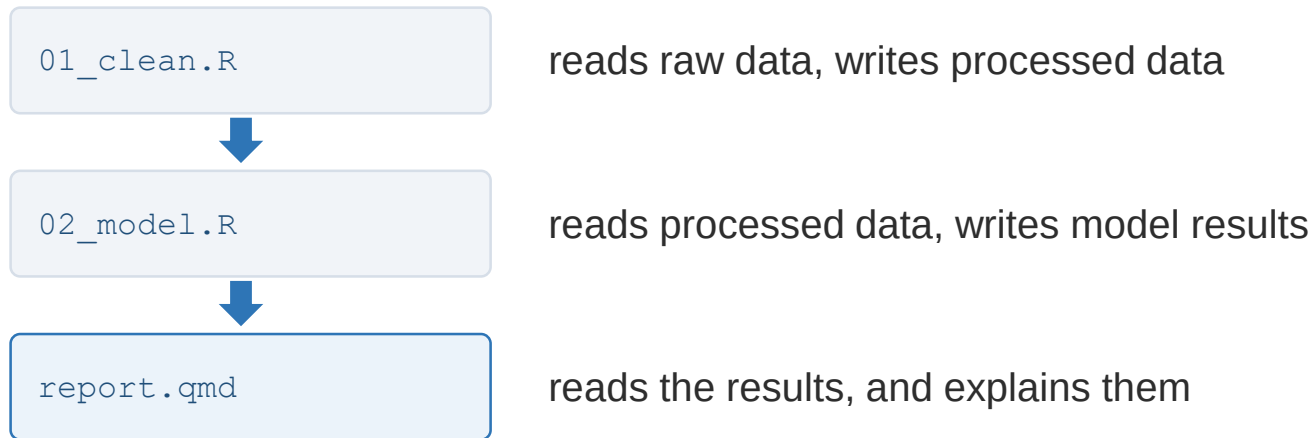
- You run chunk 3 to test something
- Then go back and change chunk 1
- The objects in memory no longer match the file
- Your screen shows results the code cannot reproduce

The cure

- Render the whole document before you trust it
- Rendering starts a fresh session with nothing loaded
- If it renders clean, the results are real
- If it fails, you just caught a bug you could not see

What is on your screen is not evidence. Only a clean render is evidence. Treat a successful render the way you would treat a passing test.

Most real projects use both: scripts do the heavy work, one document tells the story



The report does not redo the analysis — it reads the results and explains them. That keeps rendering fast, and a report nobody waits for is a report that gets rendered.

Part 3

The output

Markdown language, and how one source file becomes every report format you need

Markdown is plain text, which is why Git can actually review your writing

The whole syntax, near enough

```
# Heading          ## Subheading
**bold**           *italic*
- a bullet         1. numbered
[link](url)        `code`
| col | col |      tables
```

Why plain text wins

Reviewable: git diff shows every sentence you changed

Durable: opens in any editor, on any machine, in any year

Convertible: one source, many output formats

Small: no hidden formatting to corrupt or fight with

A Word file is a binary file: git diff on it tells you nothing at all. Markdown is the reason your writing can go through the same review process as your code.

A Quarto document has three parts: a header, your prose, and code chunks

```
---  
title: "Regional Sales Review"  
author: "Group 4"  
format: html  
---  
  
## Revenue by region  
  
Revenue grew fastest in the north.  
  
```${r}  
library(here)
Tuition <- readRDS(here("data", "process",
 "Tuition_clean.rds"))
```
```

YAML header

Between the three dashes. Title, author, and the output format.

Prose

Plain markdown. Headings, sentences, lists, links.

Code chunks

Between ```\${r}``` fences. These run when you render.

Render with the button at the top of the editor, or Ctrl/Cmd + Shift + K.

Code chunk options decide what the reader sees, without deleting any code

```
```{r}
#| echo: false
#| warning: false
#| fig-cap: "Histogram of tuition"
hist(Tuition$tuition)
```
```

echo

show the code, or hide it

eval

run the code, or skip it

warning

let warnings through, or suppress

message

same, for package messages

fig-cap

caption the figure output

Hiding code is not deleting it. The chunk still runs and still sits in the file — so the result stays reproducible even when the reader never sees how it was produced.

Numbers written into your sentences come from the code, so they cannot go stale

What you write

```
Average revenue was `r round(mean(sales$revenue))` across `r nrow(sales)` stores.
```

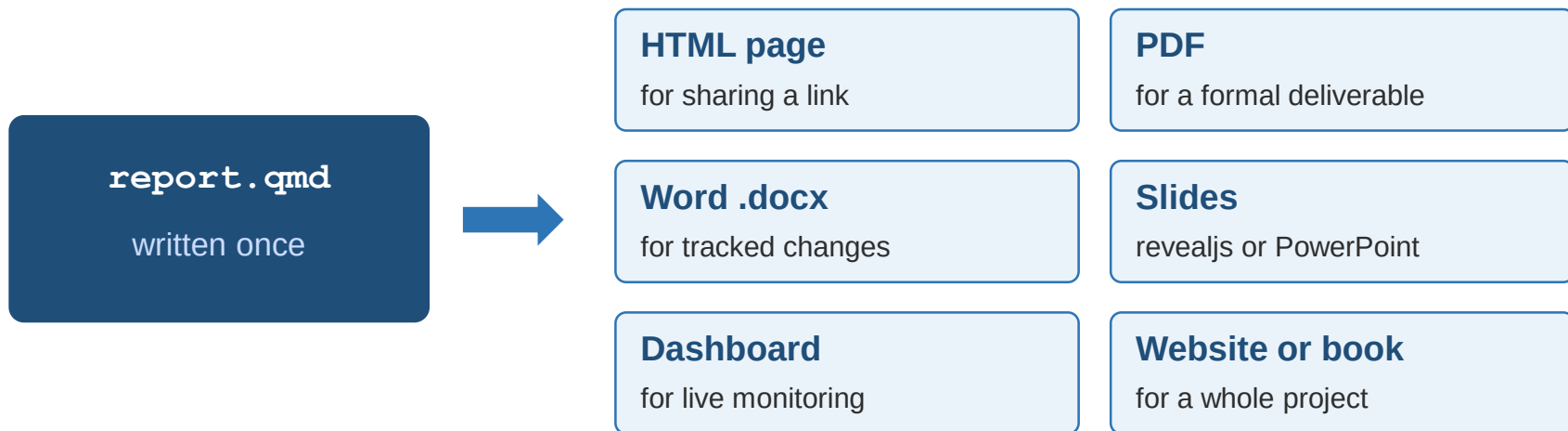
What the reader sees

Average revenue was 48,210 across 137 stores.

The client sends corrected data the night before the deadline. With typed numbers you hunt through the document for every figure you have to change. With inline code you render again, and the prose is already correct.

This is what removes the hand-copying from the diagram at the start of the lecture.

One source file renders to every format you are likely to be asked for



Your supervisor wants Word, the client wants PDF, the team wants a link, and you need slides. Same file, four renders.

Switching output format is one line in the header

One format at a time

```
format: html  
format: pdf  
format: docx  
format: revealjs  
format: pptx
```

Or several at once

```
format:  
  html: default  
  pdf: default  
  docx: default
```

From the terminal

```
quarto render report.qmd  
quarto render report.qmd --to docx
```

PDF output needs a LaTeX installation. Run `quarto install tinytex` once and it is handled.

Rendering runs everything in a fresh session, so it is also your reproducibility test

What a clean render proves

- Every file the code needs is actually in the project
- Every path resolves from the project root
- Nothing depends on objects left in your memory
- The run order in your scripts is correct

Build the habit

- Render before every push to GitHub
- If it fails, fix it before your teammates pull it
- Commit the source .qmd, gitignore the rendered output
- The output is regenerated, so it does not belong in Git

A clean render is the closest thing you have to proof that the project will run on someone else's machine. Treat a failed render as a bug you were lucky to catch.

Exercise — build the skeleton and render it three ways

Work in your project groups. 10 minutes. One project per group.

```
# make a new quarto file: File > New File > Quarto  
Document  
# save it as report.qmd  
# title: Introduce team members  
# Make sections for each member  
Each person needs to introduce your name (with  
English name, too), your habit, your social media  
(IG), and a touched photo avatar inside the report  
# and render it to multiple output formats: html,  
docx, and pptx slides  
# submit the three output files to me
```

The usual failures are a wrong file path call and a PDF render with no LaTeX installed.

Ground rules for your group project

Never edit data/raw

Every correction goes in a script

Avoid use setwd()

Open the .Rproj and use here() if need

Number your scripts

So the run order is visible

Render before you push

A failed render is a bug you caught

Commit source, not output

Rendered files are regenerated

Write the README first

One sentence, then what to run

Never editing raw data and avoid using setwd() prevent most of what goes wrong.

What to take away

- 1. Structure is a ten-minute decision with a long life.** Separate raw inputs, code, and regenerated outputs, and never edit raw data.
- 2. Scripts and documents do different jobs.** Scripts hand over data; documents hand over something a person reads. Most projects need both.
- 3. Markdown makes writing reviewable.** Plain text means Git can diff your prose, and your numbers can come from the code.
- 4. One source, every format.** HTML, PDF, Word, or slides from one file — and a clean render is your reproducibility test.

Before next week: set up your group project folder and render one report to two formats

[instructor@university.edu] · Course repo: [github.com/your-course/repo]

References and further resources

Core references

Quarto documentation. quarto.org — start with the Guide, then Using R and Output Formats.

Wickham, H., Çetinkaya-Rundel, M. & Golemund, G. (2023). R for Data Science, 2nd edition. O'Reilly. Chapters on Quarto and Quarto formats. Free at r4ds.hadley.nz

Turrell, A. (2025). Coding for Economists, chapters on Markdown, Quarto, and Reproducible Analysis. aeturrell.github.io/coding-for-economists

Posit (2026). Structuring Reproducible Research Projects in R: a workflow with renv, Quarto and GitHub. opensource.posit.co/blog

here package documentation — project-relative file paths. here.r-lib.org

renv package documentation — project-local package libraries. rstudio.github.io/renv

Bryan, J. (2017). Project-oriented workflow. rstats.wtf/project-oriented-workflow.html

Marwick, B., Boettiger, C. & Mullen, L. (2018). Packaging Data Analytical Work Reproducibly. The American Statistician, 72(1), 80-88.

Everything you need to start a project, in one place

Create the skeleton

```
library(fs)
dir_create(c("data/raw",
  "data/process", "R",
  "scripts", "outputs"))
file_create("README.md")
```

Lock the environment

```
renv::init()      # start
renv::snapshot()  # record
renv::restore()   # match
```

Paths that travel

```
library(here)
here("data", "raw", "x.csv")
# never setwd()
```

Render

```
quarto render report.qmd
quarto render report.qmd --to docx
quarto install tinytex    # for PDF
```

A suggested .gitignore: outputs/, data/raw/ (if large), renv/library/, .Rhistory, .RData, *.html, *.pdf

Six failures you will meet, and what each one actually means

"cannot open file ..."

A path is wrong. Wrap it in [here\(\)](#) and check you are inside the Project.

"object 'sales' not found"

The object lived in your memory, not in the file. The classic notebook trap: dependable file, run chunk in order.

PDF render fails on LaTeX

Run [quarto install tinytex](#) once, then render again.

"there is no package called ..."

Missing dependency. Install it [install.packages\("package_name"\)](#), then `renv::snapshot()` so others get it too.

Renders for you, not a teammate

Something is outside the project folder, or missing from `renv.lock`.

Git shows huge diffs on output

You committed rendered files. Gitignore them; commit only the source.

Read render errors from the top: the first error is the real one, and the rest are usually consequences.